What a Memory Preview Cannot Tell You: Ambiguity in AI Memory Permissions

SADIA NOMAN TONNI, United International University, Bangladesh
NAFIS FUAD SHAHID, Islamic University of Technology, Bangladesh

A memory rule states when an assistant may reuse a remembered event, and an editor can confirm a rule by previewing one use it allows and one it blocks. We ask what such a preview lets a person infer about the rule they saved. For an editor with six or twelve valid rules, we enumerate every rule compatible with a preview. Splitting the rules most evenly does not target the question a user cares about. For an explicit preference, two adaptive tests leave all three recommendation scopes open (for a one-off choice the outcome turns on a tie), while one test of transfer to another task settles the question. Weak previews are also the default. Under the intended rule, 36 of 39 possible allowed/blocked pairs contain a blocked example that no rule allows, and no pair identifies the rule. Three or four tests suffice here; when permissions depend on each other, a fixed checklist grows to six or nine and adaptive questioning needs at most five or seven. These counts use a specified rule vocabulary and fictional events. In a small pilot, 10 of 20 people who saw such a weak preview of a rule they had just chosen mispredicted, or were unsure about, at least one of two uses, and none of those 10 said the outcomes matched what they wanted. Whether better previews help people is untested.

CCS Concepts: • **Human-centered computing** → **Interactive systems and tools**; • **Security and privacy** → *Usability in security and privacy*.

Additional Key Words and Phrases: conversational memory, example-based specification, policy authoring, disambiguation

1 Introduction

A desk bought to fit a temporary workspace is useful context for a matching shelf. Should the purchase also guide a recommendation for a new home? An editor can confirm a memory rule by previewing one use it preserves and one it prevents. Suppose the preview permits the shelf and blocks the claim that the buyer generally prefers pale wood. A rule limited to the workspace and a rule allowing any task both produce those answers, so the preview hides the difference that matters to the buyer.

We ask which future requests distinguish the rules consistent with a preview. For a specified editor we enumerate the compatible rules and report three findings. (1) An even split does not target the user's question: for an explicit preference two such tests leave every recommendation scope open, and for a one-off choice a tie decides, while one test of transfer to another task settles it. (2) Weak previews are the default. Most allowed/blocked pairs an editor could show contain a blocked example that eliminates no rule. (3) Three or four well-chosen tests identify the rule exactly, and the count grows once permissions depend on each other. The enumeration is elementary; the design question is which examples a preview should contain. In a pilot with 20 people, half mispredicted or were unsure about what the rule they had just saved would do.

2 Related work

A version space is the set of hypotheses consistent with the examples seen [10], and the teaching dimension is the fewest examples that single one out [3]. Program synthesis queries distinguishing inputs [6], programming-by-example interfaces elicit such distinctions from users [8, 12, 21], Expandable Grids exposes effective permissions [14], and policy-by-example learns rules from cases [11]. We ask which contrasts a memory-rule preview must contain.

Authors' Contact Information: Sadia Noman Tonni, Department of Computer Science and Engineering, United International University, Dhaka, Bangladesh; Nafis Fuad Shahid, Department of Computer Science and Engineering, Islamic University of Technology, Gazipur, Bangladesh.

For assistant memory, user studies describe memory that departs from people's expectations [2] and disclosures that travel beyond their intended context [17]; contextual integrity frames such reuse as a flow that must fit its context [1], and benchmarks measure violations [9, 13]. Memory Sandbox lets users view and edit an agent's memories [5], other systems govern what is admitted [7, 16] or filter records by provenance and permission [18], and access-control models grant by attributes [4, 15]. The closest work is ConMem [20], which synthesizes counterfactual scenarios so that a system can audit whether to retain a memory. We use counterfactual requests for a different reader: the person checking a rule they wrote. Yan [19] found that user-authored standing policies blocked less agent overreach than per-action review, because most rules deferred to "ask". A working policy editor is not evidence of benefit.

3 Editor and diagnostic

Rules. An event records a person, a task and an evidence type: a one-off choice or an explicit preference. A rule keeps the event and sets three permissions: *recall* (may the assistant state the fact when asked), *recommendation scope* (none, the original task, or any task for the original person) and *preference use* (may the event stand as a lasting preference). Preference use requires explicit evidence and applies only to the original task. Use for another person and trait inference are always blocked, since a choice or stated preference does not establish a personality trait. This gives six valid rules for a one-off choice and twelve for an explicit preference. A request is a person, a task and a purpose (recall, recommendation, preference or trait), confirmed by the user. Two persons, two tasks and four purposes give sixteen candidate requests.

Why preview when there are settings? A preview checks whether the person's reading of a setting matches what executes. "Only this task" does not say whether a bookcase for a new apartment continues the task; a concrete request does. The enumerated rules therefore stand for the readings a person might hold: someone who takes "only this task" to cover the apartment expects what the any-task rule does.

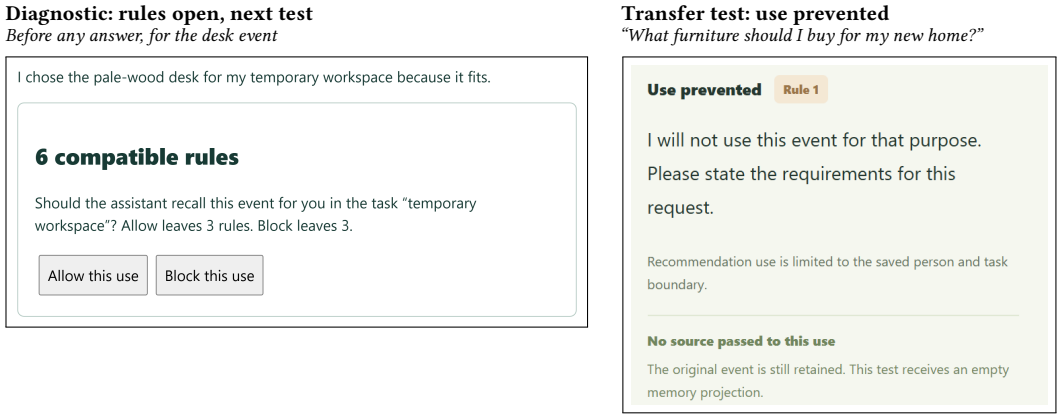


Fig. 1. Left: the diagnostic counts the rules still compatible and suggests a test. Right: the outcome of the transfer test under a saved rule limited to the original task, a request the authored previews did not include. Both are cropped from author-scripted captures.

Prototype and diagnostic. Replies are deterministic templates, and the gate agreed with a separately coded oracle on 10,000 random inputs. The diagnostic (Figure 1) keeps every rule that reproduces the answers so far and suggests the request that splits the rest most evenly (maximum entropy, uniform prior, ties in a fixed order). A separate *transfer* probe changes only the task, the desk buyer's concern.

4 What previews distinguish

Weak previews are the default. A preview pairs one request the saved rule allows with one it blocks. Under the intended one-off rule (recall on, recommendations for the original task), three requests are allowed and thirteen blocked, so an editor could show 39 previews. In 36 the blocked example is blocked under every valid rule and eliminates nothing; on average 3.2 of six rules remain, and none identifies the rule. For the intended explicit rule the figures are 44 of 48 previews and 6.3 of twelve rules. The cause is that twelve of the sixteen requests (eleven for explicit events) receive the same answer under every rule. Our own twelve authored scenarios fell into this: each prevented example was one that no rule allows, leaving four of six or six of twelve rules (Figure 2).

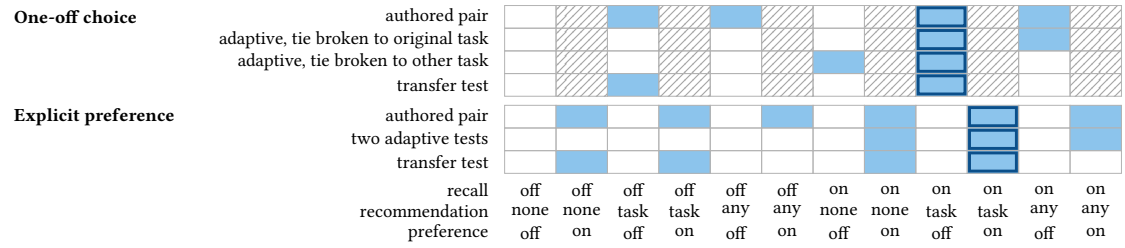


Fig. 2. Rules still compatible (shaded) after each way of testing; columns are rules, the intended rule is outlined and hatched cells are invalid for a one-off choice. Adaptive tests leave every scope for an explicit preference; for a one-off choice a tie decides.

Table 1. Two tests aimed at each intended rule: rules left K, residual bits H ($\log_2 K$; a mean for Random) and disagreement D.

	One-off choice			Explicit preference		
	K	H	D	K	H	D
Authored	4	2.0	12.5%	6	2.6	14.2%
Random	4.8	2.2	12.3%	9.0	3.1	14.1%
Adaptive	2	1.0	6.3%	3	1.6	8.3%
Transfer	2	1.0	12.5%	4	2.0	12.5%

An even split does not target the user's question. Table 1 compares four ways of choosing two tests. *Random* averages all 120 unordered pairs of requests; restricted to distinct requests that can separate rules, it leaves 2.0 and 3.8 rules. *Adaptive* takes the diagnostic's suggestion twice. *Transfer* keeps the authored preserved use and adds an original-person recommendation for another task. D is the share of remaining rule pairs that disagree, averaged over the sixteen requests. For an explicit preference the adaptive tests are recall and original-task preference, with no tie; three rules remain, one per recommendation scope (Figure 2), so transfer is entirely open, while the transfer test leaves four rules, none allowing another task. A one-off choice depends on a tie: after recall, the original-task and other-task recommendation tests split the rest equally. Our fixed order takes the first and leaves the task-limited and any-task rules; the other answers the buyer's question, with the same K, H and D.

How many tests identify a rule? Two yes-or-no answers have at most four patterns, so two tests cannot single out one of six rules. Three can: a recall request, an original-task recommendation and an other-task recommendation, all for the original person. Adding an original-task preference request identifies all twelve explicit rules. Both match the lower bound $\lceil \log_2 K \rceil$. That is one test per independent setting. Dependent permissions change this. An offline

sensitivity model makes recommendation, handoff and follow-up require recall and adds expiry (never or from day seven), preference scope (none, original or any task; explicit evidence only), a same-task handoff to one designated person and same-task follow-up; a rule permitting nothing has no expiry. This gives 25 and 77 rules. The 48 requests combine day zero or seven, two persons, two tasks and six purposes (recall, recommendation, preference, handoff, follow-up and trait). The smallest *fixed checklist*, chosen in advance to tell every two rules apart, has six and nine requests, above the bounds of five and seven; adaptive questioning needs at most five and seven.

5 Where a task ends

Can software decide whether a new request continues the task? We wrote 48 fictional request pairs (24 same task, 24 different; a new beneficiary, goal or occasion means a different task), with provisional labels drafted with AI assistance and reproduced by a second AI pass, which shows internal consistency only. On 24 held-out pairs, literal equality recognized 3 of 12 same-task pairs, a fitted text-embedding-3-small threshold accepted 23 of 24 pairs, and a label-blind Gemini 3.8 Flash judge recognized 11 of 12 with no false positive. The prototype therefore uses task identifiers the person confirms, at a cost: it blocked all 12 archived requests that renamed the task.

6 A pilot with people

Twenty adults, classmates and friends of the authors invited in person, completed an on-screen survey on the desk event. Each chose a recommendation scope (six none, nine the original task, five any task), saw a two-example preview whose blocked example no rule allows, and predicted two uses of their rule before seeing the outcomes: a lamp for the workspace and a bookcase for a new apartment. Ten of 20 mispredicted or were unsure about at least one. Five mispredicted the original task; six did not predict the transfer (three wrong, three unsure, the only unsure answers given). Errors ran both ways: four people expected a use their rule blocks and four expected a block their rule allows. None of the ten answered Yes to whether the outcomes matched what they wanted, against nine of the other ten. The sample is small and known to the authors, the median completion time was 41 seconds, and there was no comparison condition, so we report counts only. In this minimal-risk task, participants were unpaid adult volunteers who consented on screen and could stop or discard their answers; nothing identifying or sensitive was collected. We did not obtain formal ethics review; a faculty member advised that it was not required for anonymous, non-sensitive data.

7 Discussion and next study

For preview design: draw examples that some rule allows and another blocks, since an always-blocked example checks only the implementation; let the person's stated concern choose a test, since an even split can leave it open; and count the rules a preview leaves open and offer the next test, which here costs three or four tests in total.

The counts describe one finite rule set, and the diagnostic treats every rule as equally likely. Replies are templates, so nothing here shows that a language model would respect a rule. The pilot shows that readings diverge from what executes; it does not show that a better preview closes the gap. The next study should randomly show the authored or the transfer preview and compare predictions, giving people time to read.

Code and data. https://github.com/tonni19/memory-preview-ambiguity

AI use. Generative AI tools (OpenAI Codex and Anthropic Claude) were used to write the code, author the fictional fixtures and provisional labels, run the archived model calls, and draft and edit this text. The authors reviewed the manuscript and are responsible for its content.

References

[1] Adam Barth, Anupam Datta, John C. Mitchell, and Helen Nissenbaum. 2006. Privacy and Contextual Integrity: Framework and Applications. In *Proceedings of the IEEE Symposium on Security and Privacy*. IEEE, 184–198. doi:10.1109/SP.2006.32

[2] Jingruo Chen, Shuning Zhang, Eryue Xu, Jianing Li, and Xin Yi. 2026. Characterizing Memory Misalignment in Human-LLM Interaction From User Perspectives. arXiv:2609.33623. https://arxiv.org/abs/2609.33623

[3] Sally A. Goldman and Michael J. Kearns. 1995. On the Complexity of Teaching. *J. Comput. System Sci.* 50, 1 (1995), 20–31. doi:10.1006/jcss.1995.1003

[4] Vincent C. Hu, David Ferraiolo, Rick Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, and Karen Scarfone. 2014. *Guide to Attribute Based Access Control (ABAC) Definition and Considerations.* Technical Report SP 800-162. National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-162

[5] Ziheng Huang, Sebastian Gutierrez, Hemanth Kamana, and Stephen MacNeil. 2023. Memory Sandbox: Transparent and Interactive Memory Management for Conversational Agents. In *Adjunct Proceedings of UIST*. ACM, New York, NY, USA, 1–3. doi:10.1145/3586182.3615796

[6] Susmit Jha, Sumit Gulwani, Sanjit A. Seshia, and Ashish Tiwari. 2010. Oracle-Guided Component-Based Program Synthesis. In *Proceedings of ICSE*. ACM, New York, NY, USA, 215–224. doi:10.1145/1806799.1806833

[7] Chang Liu and Deliang Ding. 2026. What to Admit and How to Present: Governing Persistent Memory in LLM Agents. arXiv:2610.11188. https://arxiv.org/abs/2610.11188

[8] Mikaël Mayer, Gustavo Soares, Maxim Grechkin, Vu Le, Mark Marron, Oleksandr Polozov, Rishabh Singh, Benjamin Zorn, and Sumit Gulwani. 2015. User Interaction Models for Disambiguation in Programming by Example. In *Proceedings of UIST*. ACM, New York, NY, USA, 291–301. doi:10.1145/2807442.2807459

[9] Niloofar Mireshghallah, Neal Mangaokar, Narine Kokhlikyan, Arman Zharmagambetov, Manzil Zaheer, Saeed Mahloujifar, and Kamalika Chaudhuri. 2026. CIMemories: A Compositional Benchmark for Contextual Integrity in LLMs. In *Proceedings of ICLR*. https://proceedings.iclr.cc/paper_files/paper/2026/hash/9a2bcfaf383638e166162a25b6dff125-Abstract-Conference.html

[10] Tom M. Mitchell. 1982. Generalization as Search. *Artificial Intelligence* 18, 2 (1982), 203–226. doi:10.1016/0004-3702(82)90040-6

[11] Adwait Nadkarni, William Enck, Somesh Jha, and Jessica Staddon. 2017. Policy by Example: An Approach for Security Policy Specification. arXiv:1707.03967. https://arxiv.org/abs/1707.03967

[12] Hila Peleg, Sharon Shoham, and Eran Yahav. 2018. Programming Not Only by Example. In *Proceedings of ICSE*. ACM, New York, NY, USA, 1114–1124. doi:10.1145/3180155.3180189

[13] Sidharth Pulipaka, Oliver Chen, Manas Sharma, Taaha S Bajwa, Vyas Raina, and Ivaxi Sheth. 2026. PersistBench: When Should Long-Term Memories Be Forgotten by LLMs? arXiv:2602.01146. https://arxiv.org/abs/2602.01146

[14] Robert W. Reeder, Lujo Bauer, Lorrie Faith Cranor, Michael K. Reiter, Kelli Bacon, Keisha How, and Heather Strong. 2008. Expandable Grids for Visualizing and Authoring Computer Security Policies. In *Proceedings of CHI*. ACM, New York, NY, USA, 1473–1482. doi:10.1145/1357054.1357285

[15] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. 1996. Role-Based Access Control Models. *Computer* 29, 2 (1996), 38–47. doi:10.1109/2.485845

[16] Preeti Saraswat, Divya Neelagiri, and Ajay Manoj. 2026. Gated Memory: Admission-Controlled Memory Formation for Conversational AI. arXiv:2610.11270. https://arxiv.org/abs/2610.11270

[17] Atieh Taheri, Mahya Tazike, Patrick Carrington, and Jeffrey P. Bigham. 2026. When Disability Disclosure Travels: Memory, Privacy, and Contextual Integrity in Conversational AI. arXiv:2609.22720. https://arxiv.org/abs/2609.22720

[18] Yiqi Wang, Zihao Yan, Jiaqi Zhang, Zhangkai Wu, Mingkai Zheng, Zequn Sun, Yanming Zhu, and Taotao Cai. 2026. MAP-Graph: Provenance-Aware Shared Memory for Multi-Agent Workflows. arXiv:2608.10509. https://arxiv.org/abs/2608.10509

[19] Ting Yan. 2026. Do User-Authored Permission Policies Improve Protection Against AI Agent Overreach? arXiv:2608.27443. https://arxiv.org/abs/2608.27443

[20] Shuning Zhang, Tenghao Ji, Jingruo Chen, Eryue Xu, Bohao Chu, and Xin Yi. 2026. ConMem: Counterfactual Multi-Scenario Probing for Resolving LLM Memory Misalignment. arXiv:2610.33432. https://arxiv.org/abs/2610.33432

[21] Tianyi Zhang, London Lowmanstone, Xinyu Wang, and Elena L. Glassman. 2020. Interactive Program Synthesis by Augmented Examples. In *Proceedings of UIST*. ACM, New York, NY, USA, 627–648. doi:10.1145/3379337.3415900